MODULE III — MULTI-HOP CHAIN-BRAKING RELAY

```
import random
import time

BRAKE_THRESHOLD_G = 1.2
RSSI_ALERT_THRESHOLD_DBM = -55


class SafetyPacket:
    def __init__(self, node_id, brake_status, indicator_status, relay_flag):
        self.node_id = node_id
        self.brake_status = brake_status
        self.indicator_status = indicator_status
        self.relay_flag = relay_flag

    def __str__(self):
        return (
            f"{{ node_id: {self.node_id}, brake: {self.brake_status}, "
            f"indicator: {self.indicator_status}, relay: {self.relay_flag} }}"
        )


def read_accelerometer(simulate_brake: bool):
    if simulate_brake:
        return round(random.uniform(1.4, 2.0), 2)
    return round(random.uniform(0.0, 0.3), 2)


def simulate_rssi(in_range: bool):
    if in_range:
        return random.randint(-50, -40)
    return random.randint(-95, -80)


def run_demo():
    print("=" * 62)
    print("Smart V2V Safety Broadcast System — Module III Demo")
    print("Multi-Hop Chain-Braking Relay (software simulation)")
    print("Layout: Vehicle A --(in range)--> B --(in range)--> C")
    print("        A --(OUT OF RANGE)----------------------> C")
    print("=" * 62)

    while True:
        print("\n" + "-" * 62)
        choice = input(
            "Press [Enter] for NORMAL driving, type 'b' for Vehicle A "
            "SUDDEN BRAKING, or 'q' to quit: "
        ).strip().lower()

        if choice == "q":
            print("\nDemo ended.")
            break

        simulate_brake = choice == "b"

        accel = read_accelerometer(simulate_brake)
        brake_status = 1 if accel >= BRAKE_THRESHOLD_G else 0
        print(
            f"\n[SENSOR] Node A MPU6050 reading: {accel} g "
            f"(threshold = {BRAKE_THRESHOLD_G} g)"
        )
        print(f"[LOGIC] brake_status = {brake_status}")

        packet_a = SafetyPacket("A", brake_status, 0, relay_flag=0)
        print(f"\n[TX] Node A -> broadcasting: {packet_a}")

        if brake_status == 0:
            print("[RX] Node B receives packet, brake=0 -> no alert, no relay needed.")
            print('[OUTPUT] Node B -> Buzzer OFF | LED: GREEN | LCD: "SAFE"')
            print('[OUTPUT] Node C -> Buzzer OFF | LED: GREEN | LCD: "SAFE"')
            continue

        time.sleep(0.3)
        rssi_ab = simulate_rssi(in_range=True)
        print(f"[RX] Node B <- Node A packet received. RSSI = {rssi_ab} dBm")
```

```
        if rssi_ab >= RSSI_ALERT_THRESHOLD_DBM and packet_a.brake_status == 1:
            print("[ALERT] Node B: BRAKE detected from Node A (direct range)")
            print('[OUTPUT] Node B -> Buzzer ON | LED: RED | LCD: "BRAKE ALERT"')

            time.sleep(0.3)
            print(
                "[LOGIC] Node B: relay_flag on received packet = 0 "
                "-> relay condition met, re-broadcasting..."
            )
            packet_b_relay = SafetyPacket("A", 1, 0, relay_flag=1)
            print(f"[TX] Node B -> re-broadcasting: {packet_b_relay}")

            time.sleep(0.3)
            rssi_ac = simulate_rssi(in_range=False)
            print(
                f"\n[RX] Node C <- Node A direct signal. RSSI = {rssi_ac} dBm "
                "(below threshold; A is out of Node C's direct range)"
            )
            print("[OUTPUT] Node C ignores the original Node A signal as too weak.")

            time.sleep(0.3)
            rssi_bc = simulate_rssi(in_range=True)
            print(f"[RX] Node C <- Node B relayed packet. RSSI = {rssi_bc} dBm")

            if (
                rssi_bc >= RSSI_ALERT_THRESHOLD_DBM
                and packet_b_relay.brake_status == 1
            ):
                print("[ALERT] Node C: BRAKE alert received via RELAY from Node B")
                print(
                    '[OUTPUT] Node C -> Buzzer ON | LED: RED | '
                    'LCD: "RELAY ALERT"'
                )
            else:
                print("[OUTPUT] Node C receives a weak relayed signal; no alert.")
        else:
            print("[OUTPUT] Node B: signal too weak or no brake event; no alert or relay.")


if __name__ == "__main__":
    run_demo()
```